ChatGPT

End-to-End Scalable Time Series Forecasting System

Conceptual blueprint of an ML-based forecasting pipeline, from data ingestion to model deployment and continuous forecasting [1] [2].

Building a scalable end-to-end forecasting system involves **many components** – from data collection and storage, to model development, deployment, and continuous operation. Below we break down each part of the pipeline and discuss what you need (including code concepts) for implementing a robust forecasting system for **time series data** (e.g. power demand and prices at 5min, 15min, 30min, or hourly intervals). We'll also note any elements missing from your list and how to execute this project on the **cloud** for multiple regions using libraries like LightGBM, scikit-learn, and SVM.

1. Data Collection and Ingestion

The first step is to **collect relevant data** and ingest it into your system. This may involve scraping or fetching data from various sources (internal databases, APIs, web pages, etc.). Ensure you gather not only the primary time series (e.g. historical power demand/prices) but also any **external factors** that could improve forecasts (weather data, calendar events/holidays, etc.) [3]. For example, if forecasting power demand, you might pull temperature forecasts or holiday schedules as additional features.

Data scraping can be done with scripts using libraries like **Requests** (for APIs or HTML) and **BeautifulSoup** (for HTML parsing) or specialized APIs provided by data vendors. For instance, to fetch data from an API and load into a pandas DataFrame:

```
import requests
import pandas as pd

url = "https://api.operator.com/power_demand?region=XYZ&interval=5min"
response = requests.get(url)
data_json = response.json()
df = pd.DataFrame(data_json['results'])  # adjust based on actual JSON structure
```

If data comes in files (CSV, etc.), you could use **pandas** to read them directly. The key is to automate this collection so it can run regularly.

2. Data Storage in a SQL Database

Once data is collected, store it in a **central database** for reliability and scalability. A PostgreSQL database is a good choice for time series data storage (you might also consider time-series databases, but PostgreSQL

works fine and integrates well with analytics libraries). Design a schema that captures the time series and related info – for example, a table with columns: `timestamp`, `region`, `value`, and possibly columns for any raw features or source identifiers.

Using **SQLAlchemy** or a PostgreSQL adapter like `psycopg2`, you can write data to the database. For example, using pandas with SQLAlchemy for convenience:

```
from sqlalchemy import create_engine

# Create connection to Postgres (ensure to replace with actual credentials/host)
engine = create_engine("postgresql://username:password@host:port/dbname")

# Write the DataFrame to a SQL table (append new data)
df.to_sql("raw_demand_data", engine, if_exists="append", index=False)
```

This will store the scraped data in a table. Storing data in SQL allows easy querying and is essential for an end-to-end pipeline (connects the data to the modeling) [4]. Make sure to **index** important fields (like timestamp, region) in the database for faster retrieval.

3. Data ETL and Processing Pipeline

Having raw data in the database, the next component is an **ETL (Extract, Transform, Load) pipeline** to transform raw data into model-ready form. This includes pulling data from the database, cleaning it, handling missing values or outliers, and merging in any external data (like matching each timestamp with a temperature or holiday flag, if those are stored separately).

You can implement the ETL in Python using pandas or an ETL framework. For example:

```
# Extract: read raw data from the database
query = "SELECT timestamp, region, value, ... FROM raw_demand_data WHERE region
= 'XYZ';"
df = pd.read_sql(query, engine, parse_dates=['timestamp'])

# Transform: basic cleaning
df = df.sort_values('timestamp')
df = df.dropna()            # drop or impute missing values
df = df[df['value'] >= 0]   # basic sanity check, remove negative demand if
invalid

# (Optional) Merge external features (e.g., weather data)
weather = pd.read_sql("SELECT timestamp, temp FROM weather_data WHERE
region='XYZ';", engine)
df = pd.merge_asof(df, weather, on='timestamp', direction='nearest')
```

If needed, apply any data quality checks in this stage – for instance, remove obvious anomalies or fill them with suitable values (you might flag and treat anomalous spikes so they don't skew the model training). This corresponds to the **"automatic data preparation"** step in a forecasting blueprint (correlating datasets, handling anomalies, etc.) [5].

After transformation, **load** the processed data into a **feature store** or a separate schema/table. For example, you might create a table `model_features` to store the features that will be used by the model. Storing processed features can help if you want to reuse them or quickly retrieve them for inference later.

4. Automated Data Refresh Scheduling

A scalable system needs **constant automated data refreshes** so that forecasts stay up to date. Set up a **schedule** to periodically run the data collection and ETL pipelines. This could be as simple as a cron job or a scheduled task (e.g., using **Apache Airflow**, **AWS Step Functions**, or cloud scheduler services).

For example, if your data frequency is 5 minutes, you might schedule the pipeline to run every 5 minutes to fetch the latest data and update the database. For hourly frequency, run it hourly, etc. The system should be configurable for different frequencies (5min, 15min, 30min, 1hr) – perhaps by having a configuration that sets the schedule and aggregation level.

On the cloud, you might use **AWS CloudWatch Events** (EventBridge) or **GCP Cloud Scheduler** to trigger a **Lambda function** or a container that runs your pipeline code on schedule. Ensure the pipeline is **idempotent** (running it twice in a row doesn't corrupt data) – for example, only pull new data since the last timestamp you have, and avoid duplicate inserts.

Automating this ensures your pipeline continuously ingests new data without manual intervention, which is crucial for a production-grade forecasting system.

5. Feature Engineering for Time Series

With clean data ready, you need to create features that help your models learn patterns. **Feature engineering** in time series is critical – include not just the raw time series values, but also derived features capturing trends and seasonality:

- **Lag features**: prior values of the time series (e.g., value 1 step ago, 3 steps ago, 1 day ago at the same time). For example, `df['lag_1'] = df['value'].shift(1)`. These give the model a sense of recent history.
- **Rolling window statistics**: moving averages, sums or standard deviations over past windows (e.g., mean of last 12 hours, max of last 7 days). This captures longer-term trends or volatility. For instance: `df['rolling_24h_mean'] = df['value'].rolling(window=288).mean()` if 5-min interval (288 points = 24h).
- **Date/time attributes**: from the timestamp, derive hour of day, day of week, month, etc., often as one-hot or cyclical encoding. E.g., `df['hour'] = df['timestamp'].dt.hour` to capture daily patterns, or use sine/cosine transform for cyclic features.

- **External regressors**: incorporate known future variables like weather forecasts, holiday flags, special events, etc., which can significantly improve accuracy [3]. For power demand, temperature is vital; for prices, fuel costs or demand could be relevant.
- **Region-specific features**: If modeling multiple regions, some regions might have unique patterns (e.g., industrial vs residential load). You might include region as a feature in a global model, or more typically, **build separate models per region**. Given the question, you likely will train a different model per region (hence "multiple different models per region").

It's wise to write reusable **feature engineering functions** that take a raw time series DataFrame and return a DataFrame of features. This ensures consistency between training and inference (you'll use the same feature logic when making new forecasts). For example:

```
def make_features(df):
    # df is expected to be sorted by timestamp and continuous
    df_feat = df.copy()
    # Lag features
    df_feat['lag_1'] = df_feat['value'].shift(1)
    df_feat['lag_12'] = df_feat['value'].shift(12)  # e.g., 1 hour ago if 5-min
data
    # Rolling window
    df_feat['rolling_1h_mean'] = df_feat['value'].rolling(window=12).mean()
    # Time features
    df_feat['hour'] = df_feat['timestamp'].dt.hour
    df_feat['day_of_week'] = df_feat['timestamp'].dt.dayofweek
    # ... more features ...
    return df_feat.dropna()  # drop initial NaNs from lag/rolling
```

Apply such functions in your ETL pipeline (or at model training time) to enrich the data. Good feature engineering will feed the ML algorithms the information needed to learn complex patterns.

6. Model Training with Cross-Validation and Hyperparameter Tuning

Now comes model training. You listed LightGBM, scikit-learn, and SVM as desired libraries – these can all be used for forecasting by treating it as a regression problem on the features we created. Common choices include **LightGBM** (gradient boosting decision trees, which is fast and handles large data well) and various regression models from **sklearn** (like linear regression, random forest, or even Support Vector Regressors). You might even ensemble them or test several algorithms to see which works best for each region.

To train robust models, follow **best practices**: use **out-of-sample cross-validation** that respects time series order. This means we **never train on future data and test on past data**, which would lead to leakage [6]. Instead, use techniques like **walk-forward validation** (also called rolling origin or time series CV). In scikit-learn, you can use `TimeSeriesSplit` for this purpose – it provides sequential train/test splits on time-ordered data [6]. Each split trains on an initial segment and tests on the next segment, then expands or rolls forward for the next fold. This approach ensures each fold's test simulates a "future" that wasn't seen in training [7]. For example:

```
from sklearn.model_selection import TimeSeriesSplit, GridSearchCV

X = features_df.drop(columns=['value'])  # feature matrix
y = features_df['value']                 # target variable

tscv = TimeSeriesSplit(n_splits=5)
model = lgb.LGBMRegressor()  # LightGBM model (as an example)

param_grid = {
    'num_leaves': [31, 63],
    'learning_rate': [0.1, 0.01],
    # ... other hyperparameters ...
}
grid_search = GridSearchCV(model, param_grid, cv=tscv,
scoring='neg_mean_absolute_error')
grid_search.fit(X, y)
print("Best params:", grid_search.best_params_)
best_model = grid_search.best_estimator_
```

In this snippet, `TimeSeriesSplit` will create 5 folds, each time training on an expanding window and testing on the subsequent chunk of data. **Rolling vs. expanding window**: by default, `TimeSeriesSplit` uses an expanding window (each training fold includes all data up to that point) [8]. If you prefer a rolling window (fixed size training set moving forward in time), you can set a `max_train_size` in `TimeSeriesSplit` or manually slice your data in a loop to simulate a sliding window. The pipeline should be flexible to do either, since sometimes you want to limit training size (to give equal weight to recent data) or use all data.

During cross-validation, evaluate each model's performance (e.g., use MAE, RMSE or MAPE metrics) to ensure the model generalizes well to unseen time periods. This serves as your **out-of-sample CV pipeline** for model selection.

Hyperparameter tuning can be done via grid search as above, or more efficiently via random search or Bayesian optimization (libraries like **Optuna** or **Hyperopt**). The goal is to find the best parameters for each model/region. For example, tuning LightGBM's tree parameters or an SVM's kernel parameters with time-series CV. The **optimal parameters** found should be saved (so you know what settings the final model uses) – you can log them or store in a config file or database.

By using walk-forward (rolling) validation in the tuning process, you ensure the chosen model and hyperparameters are truly optimal for predicting forward in time [7]. Myst AI's platform, for instance, emphasizes this kind of backtesting to ensure models will perform well in deployment [7]. They note that *walk-forward cross-validation* is crucial because it "ensures future data never leaks into training" and closely mimics deployed performance [7].

Also consider training multiple model types and comparing. In forecasting, often an **ensemble** of models can perform better than any single model [9] [10]. You might ultimately select one model per region (whichever had best validation score), or even combine predictions (though that adds complexity).

7. Backtesting with Rolling Forecasts (Out-of-Sample Evaluation)

Beyond cross-validation during training, it's often useful to do a **backtest** of the entire forecasting process. This means taking historical data and simulating how your model would have forecasted in the past on a rolling basis. In practice, you likely did this with the CV, but you can also explicitly perform an **out-of-sample rolling forecast** evaluation:

- Choose a cutoff date in the historical data. Train the model on data before that date.
- Forecast from that date forward N steps (e.g., predict the next day or week).
- Compare the forecast to the actual values (which you withheld).
- Then roll the window forward (add that next period into training, or move the window) and repeat for subsequent periods.

This is essentially what TimeSeriesSplit automates, but you can also implement it manually for full control (especially if you want to test expanding vs fixed window). The difference between **expanding window** and **rolling window**: expanding keeps adding new data to the training set (growth over time), whereas rolling window keeps the training set size constant by dropping the oldest data as new data comes in. You might try both to see which gives better performance for your domain.

The backtest process should mirror how you will use the model in production, which gives a realistic measurement of performance. For example, if you plan to retrain weekly and forecast daily, simulate that in backtesting. Ensure all results (e.g., error metrics from each backtest window) are recorded. This not only validates model accuracy but also provides a baseline to monitor future performance. As Anodot's guide suggests, comparing forecasts with actuals over a historical holdout is vital for evaluating accuracy [11].

8. Model Selection and Saving the Optimal Model

After tuning and validation, you should have an **optimal model** (or set of models) for each forecasting task (e.g., one per region). The next step is to **persist the model** for future use. This involves saving the model object and any associated preprocessing so you can load it later to make predictions.

For scikit-learn models or LightGBM, you can use Python's joblib or pickle to save models:

```
import joblib
joblib.dump(best_model, f"model_region_{region_name}.pkl")
```

This will serialize the model (including learned parameters). LightGBM's `Booster` also has a native `save_model()` method which can output a text file representation. Use whichever is convenient, but **do store the model** – either on disk, or even better, in a **cloud storage** (like AWS S3) or a model registry. Storing in cloud storage is good for a cloud-based system so that any compute instance can load the model. If using multiple models (one per region), you might organize them by region and timestamp or version.

Along with the model, save any metadata needed to use it: - The model's hyperparameters and training data range. - The feature engineering steps or scalers used (if you normalized data or need to apply same

transformations to new data, save those objects too). - The performance metrics on training/validation (for reference). - A version number or timestamp for the model.

Persisting models makes them **reusable** for inference without retraining every time [12] – essentially you have a "customized forecast model" that can be run on demand [12].

9. Loading Models and Forecasting on New Data

With trained models saved, the system should support **loading the model and running forecasts** whenever new data arrives. In production, this typically means: when the latest actual data is available up to time *T*, you load the model, construct the features for time *T* (and prior as needed), and predict future values *T+1, T+2, ... T+N*.

To do this: 1. **Retrieve the latest data** from the database (up to current time). 2. Create the feature matrix for the next N steps. Often, forecasting is done iteratively: predict one step ahead, then append that prediction to the input (as if it were an actual value) to predict the following step, and so on. This is needed if your features include past values or rolling stats – you have to update those as you move forward. 3. **Load the model** from storage (using `joblib.load` or similar). 4. Call the model's `.predict()` on your feature matrix to get forecasts.

For example, a simple iterative forecast using a trained model for N=3 steps ahead:

```
import joblib
model = joblib.load("model_region_X.pkl")

latest = fetch_latest_data(region="X", last_n_points=1000)  # get recent data
for features
features = make_features(latest)  # create features up to current time T
current_features = features.iloc[-1]  # features for time T (the latest known
time)

preds = []
current_input = current_features.copy()
for step in range(3):  # predict 3 future intervals
    y_pred = model.predict([current_input.values])[0]
    preds.append(y_pred)
    # Update current_input for next prediction step:
    current_input = update_features_for_next_step(current_input, y_pred)
```

In the above pseudocode, `update_features_for_next_step` would handle shifting the window: e.g., set `lag_1` = last actual value becomes now y_pred, update timestamp to next interval, recompute time features (hour/day) for the new timestamp, etc. Essentially, after each forecast, you prepare the feature row for the next timestamp as if the prediction were actual. Some libraries (like statsforecast or darts) can automate recursive forecasting; otherwise, a manual loop works.

If your forecast horizon is small, this iterative approach is fine. If you need a longer horizon or more complex method, you could train a **direct multi-step model** (e.g., one model predicts 6 steps ahead in one go), but that's more complex and typically requires more output handling. Iterative forecasting with a single-step model is common and aligns with using sklearn/LightGBM.

After obtaining the forecast values, **store the forecasts** back to the database or wherever needed. You might have a table like `forecast_results` with columns: `timestamp (of forecasted period)`, `region`, `forecast_value`, `model_version`, `created_at` (when the forecast was made). Storing forecasts allows you to later join them with actuals to evaluate performance or to feed into dashboards. It also provides traceability (what was predicted vs what happened).

Finally, **consume the forecasts** – in a real system, this could mean displaying them on a dashboard, sending alerts if certain thresholds are met, or providing them via an API to other services [2]. In your question, you focus on storing them, but do plan for how the forecasts will be used by end users or systems (this is the last mile of an ML pipeline: delivering the insight) [2].

10. Calculating and Storing Performance Metrics

To close the loop, the system should evaluate how well the forecasts are doing. This involves computing **error metrics** by comparing forecasts to actuals once the actuals become available. Common metrics for time series forecasting include MAE (Mean Absolute Error), RMSE (Root Mean Squared Error), MAPE (Mean Absolute Percentage Error), etc.

For every forecast cycle, you can calculate metrics for that horizon when actual data comes in. For example, if you forecasted day 2025-09-24 and now the actual data for that day is available, compute the error:

```
from sklearn.metrics import mean_absolute_error, mean_squared_error

# Retrieve forecast and actuals for the period of interest
forecast = get_forecast(region="X", date="2025-09-24")
actuals = get_actuals(region="X", date="2025-09-24")

mae = mean_absolute_error(actuals, forecast)
rmse = mean_squared_error(actuals, forecast, squared=False)
mape = np.mean(np.abs((actuals - forecast) / actuals)) * 100
```

Store these metrics in a table (e.g. `forecast_metrics`) with identifiers for model, region, timestamp, etc. Monitoring these metrics over time is important to detect if the model's performance is degrading.

In a scalable setup, you might automate this metric calculation as part of the daily pipeline – e.g., every time new actual data is ingested, compute yesterday's forecast error. This provides **monitoring** of model accuracy in production. As one MLOps article notes, continuous monitoring (estimated vs realized performance) is key to know if models are still performing well [13] [14].

11. Automated Model Retraining and Updating

Your question rightly includes **automated refitting** of models when new data arrives. Over time, as more data is collected (or if the underlying patterns change), the model should be retrained to stay up-to-date. There are two common strategies here: - **Scheduled retraining**: e.g., retrain the model daily, weekly, or monthly with the latest data. This ensures the model incorporates the newest patterns. Many production systems do this on a regular cadence [15]. - **Performance-triggered retraining**: monitor the error metrics; if error exceeds a certain threshold or if you detect **data drift** (the distribution of inputs shifts), trigger a retraining. This is sometimes called intelligent retraining – only update when needed [15].

At minimum, a fixed schedule (say, retrain every week) is a straightforward approach. Given power demand/price might have seasonal changes, you could schedule retrains more frequently if data is non-stationary. Automated retraining can be set up in your pipeline scheduler (e.g., an Airflow DAG could have a weekly task to retrain models).

When retraining, you'll essentially repeat the training process with all or most of the data available. You might choose a rolling window of recent data for training (if you want to forget very old data) or always use all data (if older patterns still matter – for long seasonal cycles). After retraining: - Evaluate the new model (using the CV/backtest procedure). - If it's better or at least not worse than the old model, **promote** it to production (save it and use it for future forecasts). - Archive the old model (or at least keep its artifact and metrics). This is important for **auditability** and potential rollback if the new model has issues [16].

In a cloud deployment, you might automate this with an MLOps pipeline. For instance, using **AWS SageMaker**, you can script a pipeline that triggers on new data, trains a model, and then deploys it. Or use CI/CD style workflows where model training jobs are triggered and results are validated before going live. The Myst AI platform highlights the benefit of **continuous retraining (on a user-defined schedule)** as a best practice to maintain accuracy over time [16].

Don't forget to update any references to the model in your forecasting job to use the new model version. One robust way is to not overwrite the model file but rather write models with versioned names (like `model_regionX_v2.pkl`), and have a metadata store or config that points to the "current" model for each region.

12. Cloud Deployment and Scalability Considerations

Since you want a **cloud-based** system, consider the following to make it scalable and robust: - **Infrastructure**: Use managed services where possible. For example, an AWS RDS or GCP Cloud SQL for PostgreSQL to store data; S3 or Blob storage for large data files and model artifacts; a compute service like AWS EC2, AWS Batch, or Kubernetes cluster to run training jobs especially if they are heavy. You might also use serverless functions for lightweight tasks (like triggering a scrape or running a quick prediction), but training a LightGBM on large data might need a beefier environment. - **Parallelism**: Since you will have multiple models (one per region), design the training and forecasting steps to run in parallel where possible. For example, if using Airflow or another workflow manager, you could have tasks for each region's model that run concurrently. On Kubernetes or cloud, you could spin up separate pods/jobs for each region's training. This ensures the system scales horizontally as you add regions. - **Monitoring and Logging**: Include logging at each step (data ingestion, training, forecasting) to track what's happening. Set

up alerts if data pipelines fail or if forecast errors spike (e.g., if MAE goes above a threshold, send a notification). This way, you can catch issues early. Monitoring isn't explicitly in your list, but it's a vital piece of an end-to-end system – knowing whether the pipeline ran successfully and the models are performing is crucial for reliability [13] [14]. - **Security and Access**: Ensure credentials for your database and APIs are stored securely (use vaults or cloud secrets managers). Control access to the model and forecast results if needed (since this is potentially sensitive data). - **Cost Management**: With frequent retraining and possibly high data frequency, be mindful of cost. If you retrain very often or for many regions, using spot instances or efficient algorithms (LightGBM is quite efficient; SVM can be slower on large data) will help. One strategy from industry is to retrain models only when needed (for example, only when performance drops beyond a point) to save on cloud costs [15], rather than a rigid schedule that might be overkill.

13. Additional Considerations (What's Missing?)

Your outlined pipeline is quite comprehensive. A few additional components to consider adding to your **end-to-end ML system** are:

- **Data and Feature Validation**: Before training or forecasting, have checks to ensure the data is complete and within expected ranges. This prevents feeding bad data to the model (which could produce bad forecasts).
- **Model Versioning & Experiment Tracking**: Use a tool or system (like MLflow, DVC, or even just a structured naming scheme) to version-control your models and datasets. This allows you to reproduce results and roll back if a new model version underperforms.
- **Ensemble or Benchmark Models**: Sometimes it's useful to include a simple baseline forecast (e.g., "yesterday's value" or a moving average) to benchmark your ML models. This wasn't mentioned but is good for context – your advanced models should beat the baseline; if not, re-examine them.
- **Forecast Consumption**: Plan how forecasts will be used. Storing in a DB is good, but ensure there's an interface for users or systems to retrieve them. This could be a lightweight API or a dashboard that reads from the DB. An API could allow, for example, a web service to request the latest forecast for region X, and your system queries the DB and returns it.
- **Error Analysis and Model Improvement**: Incorporate periodic review of errors (for example, if certain days consistently have high error, maybe there's an unmodeled effect like a special event). This feedback loop can guide adding new features or choosing a different model if needed (continuous improvement).
- **Scale to New Regions or Frequencies**: Since the system is parameterizable by region and frequency, document clearly how one would add a new region or change the forecast interval. Ideally it's as simple as updating a config and the pipeline can handle it (for instance, frequency could affect how features are generated and what schedule to use).

By covering these aspects, you ensure nothing critical is missing. In summary, a successful time series forecasting system connects and preps data, trains robust models (with proper validation) for each series, deploys and automates those models to produce forecasts on schedule, and monitors the results for quality [17] [2]. With the steps above and the use of powerful ML libraries (LightGBM, sklearn, etc.) [18], you can execute this project and build a scalable forecasting system. It's a complex project, so approach it iteratively: start with a simple pipeline, get it working end-to-end, then refine each component (better features, more automation, cloud optimizations) as you progress.

Sources:

- Ira Cohen, *"The Key Principles of a Successful Time Series Forecasting System"* – describes the blueprint of connecting data, preparing it, training ML models, creating a persistent model, and continuously forecasting and delivering results [17] [2].
- Myst AI Blog, *"Delivering accurate forecasts to energy companies"* – emphasizes using the best ML libraries (sklearn, LightGBM, etc.) and walk-forward cross-validation to avoid data leakage [18] [7], plus automating hyperparameter tuning and continuous retraining for time series models [19] [16].
- Vechtomova Maria, *"Monitoring and retraining strategies for demand forecasting models"* – discusses model performance monitoring and strategies like fixed schedule vs. trigger-based retraining to keep forecasts accurate over time [15].
- Scikit-learn Documentation – on using `TimeSeriesSplit` for time series cross-validation [6] [8], ensuring training on past and testing on future data (no leakage).

---

[1] [2] [3] [4] [5] [9] [10] [11] [12] [17] Time Series Forecasting: The Key Principles of a Successful System for Business
https://www.anodot.com/blog/time-series-forecasting/

[6] [8] TimeSeriesSplit — scikit-learn 1.7.2 documentation
https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html

[7] [16] [18] [19] How the Myst Platform delivers highly accurate forecasts to energy companies | by Erin Boyle | Myst AI
https://blog.myst.ai/how-the-myst-platform-delivers-highly-accurate-forecasts-to-energy-companies-8fb75387f3ec?gi=6ef0eddfc3f8

[13] [14] [15] Monitoring and retraining strategies for demand forecasting models | by Vechtomova Maria | Marvelous MLOps | Medium
https://medium.com/marvelous-mlops/monitoring-and-retraining-strategies-for-demand-forecasting-models-0983af530ecf